

MUNICIPAL COMPLAINT SYSTEM

DevOps Deployment Guide

Local → ECR → EKS | Terraform + Jenkins CI/CD

Repo	https://github.com/mantu0tech/Complain_System.git
Region	ap-south-1 (Mumbai)
Stack	Flask + Gunicorn + PostgreSQL RDS + S3 + SNS
Deployment	EKS via Terraform + Jenkins pipeline
Image Registry	Amazon ECR

DOCUMENT 1

All Commands — Copy-Paste Reference

Replace all placeholders before running: YOUR_ACCOUNT_ID, YOUR_JUMP_SERVER_IP, YOUR_BUCKET_NAME, YOUR_RDS_HOST, YOUR_VPC_ID, YOUR_DB_PASSWORD

Section 1 — AWS Pre-Setup (Run from Local Machine)

Verify AWS CLI

```
aws s3 ls
aws sts get-caller-identity
```

Create S3 Bucket

```
aws s3 mb s3://YOUR_BUCKET_NAME --region ap-south-1
```

Create SNS Topic

```
aws sns create-topic --name demo --region ap-south-1
# Save the ARN from output -> use as SNS_TOPIC_ARN in .env
```

Subscribe Email to SNS

```
aws sns subscribe \
  --topic-arn arn:aws:sns:ap-south-1:YOUR_ACCOUNT_ID:demo \
  --protocol email \
  --notification-endpoint your@email.com \
  --region ap-south-1
```

Create ECR Repository

```
aws ecr create-repository \
  --repository-name complain-system \
  --region ap-south-1
```

Section 2 — Docker (Local Windows)

Build and Start

```
docker compose up --build
```

Create DB Tables (if needed)

```
docker exec -it CONTAINER_ID python -c \  
    "from app import app, db; app.app_context().push(); db.create_all();"   
print('Tables created')
```

Useful Docker Commands

```
docker ps                # list running containers   
docker compose down      # stop all containers   
docker logs CONTAINER_ID # view logs
```

Section 3 — Terraform

Navigate and Initialize

```
cd Complain_System/terraform/municipal   
terraform init
```

Plan and Apply

```
terraform plan   
terraform apply --auto-approve
```

EKS creation takes ~15 minutes. Do not interrupt.

Get Outputs

```
terraform output   
terraform output database_url      # get sensitive DB URL
```

Destroy (When Done)

```
terraform destroy --auto-approve
```

Section 4 — Jump Server Setup

SSH Into Server

```
ssh -i ~/.ssh/municipal-key.pem ubuntu@YOUR_JUMP_SERVER_IP
```

Install Tools

```
sudo apt install awscli -y   
git clone https://github.com/mantu0tech/Complain_System.git   
cd Complain_System
```

```
bash install.sh
```

Verify Installation

```
aws --version  
docker ps  
kubectl version --client  
eksctl version
```

Connect kubectl to EKS

```
aws eks update-kubeconfig --name municipal-dev --region ap-south-1  
kubectl get nodes
```

Check Who Has EKS Access

```
aws eks list-access-entries --cluster-name municipal-dev --region ap-south-1
```

Section 5 — Build & Push to ECR

ECR Login

```
aws ecr get-login-password --region ap-south-1 | \  
  docker login --username AWS --password-stdin \  
  YOUR_ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com
```

Build, Tag, Push

```
git checkout devops  
docker build -t complain-system .  
docker tag complain-system:latest \  
  YOUR_ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com/complain-system:latest  
docker push \  
  YOUR_ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com/complain-system:latest
```

Verify in ECR

```
aws ecr describe-images --repository-name complain-system --region ap-south-1
```

Section 6 — Kubernetes Manifests

Apply in Order

```
cd ~/Complain_System/k8s
```

```
kubectl apply -f namespace.yml
kubectl apply -f secret.yml
kubectl apply -f deployment.yml
kubectl apply -f service.yml
kubectl apply -f ingress.yml
```

Verify

```
kubectl get all -n municipal
kubectl get pods -n municipal
kubectl get ingress -n municipal
```

Debug Commands

```
kubectl logs POD_NAME -n municipal
kubectl describe pod POD_NAME -n municipal
kubectl exec -it -n municipal POD_NAME -- printenv | grep DATABASE
```

Section 7 — AWS Load Balancer Controller

Add Helm Repo

```
helm repo add eks https://aws.github.io/eks-charts
helm repo update
```

Get VPC ID

```
aws eks describe-cluster \
  --name municipal-dev \
  --region ap-south-1 \
  --query "cluster.resourcesVpcConfig.vpcId" \
  --output text
```

Save the vpc-xxxxxxxx value shown — you need it below.

Download and Create IAM Policy

```
curl -o iam_policy.json \
https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/iam_policy.json
aws iam create-policy \
  --policy-name AWSLoadBalancerIAMPolicy \
  --policy-document file://iam_policy.json
```

Associate OIDC Provider

```
eksctl utils associate-iam-oidc-provider \
```

```
--cluster municipal-dev \  
--region ap-south-1 \  
--approve
```

Create IAM Service Account (10-15 min)

```
eksctl create iamserviceaccount \  
  --cluster municipal-dev \  
  --namespace kube-system \  
  --name aws-load-balancer-controller \  
  --attach-policy-arn arn:aws:iam::YOUR_ACCOUNT_ID:policy/AWSLoadBalancerIAMPolicy \  
  --override-existing-serviceaccounts \  
  --approve \  
  --region ap-south-1
```

Install via Helm

```
helm install aws-load-balancer-controller eks/aws-load-balancer-controller \  
  -n kube-system \  
  --set clusterName=municipal-dev \  
  --set serviceAccount.create=false \  
  --set serviceAccount.name=aws-load-balancer-controller \  
  --set region=ap-south-1 \  
  --set vpcId=YOUR_VPC_ID
```

Verify and Get App URL

```
helm list -n kube-system  
kubectl rollout restart deployment aws-load-balancer-controller -n kube-system  
kubectl get pods -n kube-system  
kubectl get ingress -n municipal      # ADDRESS = your public app URL  
kubectl logs -n kube-system deployment/aws-load-balancer-controller
```

Section 8 — Jenkins CI/CD

Install Jenkins

```
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | \  
  sudo tee /usr/share/keyrings/jenkins-keyring.asc > /dev/null  
echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \  
  https://pkg.jenkins.io/debian-stable binary/" | \  
  sudo tee /etc/apt/sources.list.d/jenkins.list  
sudo apt update -y && sudo apt install -y openjdk-17-jdk jenkins  
sudo systemctl enable jenkins && sudo systemctl start jenkins
```

Allow Jenkins to Use Docker and kubectl

```
sudo usermod -aG docker jenkins
sudo mkdir -p /var/lib/jenkins/.kube
sudo cp ~/.kube/config /var/lib/jenkins/.kube/config
sudo chown -R jenkins:jenkins /var/lib/jenkins/.kube
sudo systemctl restart jenkins
```

Get Initial Admin Password

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
# Open: http://YOUR_JUMP_SERVER_IP:8080
```

Verify Jenkins Can Access AWS

```
sudo -u jenkins aws sts get-caller-identity
sudo -u jenkins aws ecr describe-repositories --region ap-south-1
sudo -u jenkins kubectl get nodes
```

Section 9 — .env File Template

On EC2/EKS never add `AWS_ACCESS_KEY_ID` — the IAM role handles auth automatically.

```
# App
SECRET_KEY=your-super-secret-key-change-this
```

```
# Database — use RDS endpoint on EKS (NOT db:5432)
DATABASE_URL=postgresql://postgres:YOUR_DB_PASSWORD@YOUR_RDS_HOST:5432/complaints_db
```

```
# AWS
AWS_REGION=ap-south-1
BUCKET_NAME=your-s3-bucket-complain
SNS_TOPIC_ARN=arn:aws:sns:ap-south-1:YOUR_ACCOUNT_ID:demo
```

```
# Local dev only — REMOVE on server
# AWS_ACCESS_KEY_ID=your-key
# AWS_SECRET_ACCESS_KEY=your-secret
```

Section 10 — Cleanup

Delete IAM Service Account First

```
eksctl delete iamserviceaccount \
  --cluster municipal-dev \
  --namespace kube-system \
```

```
--name aws-load-balancer-controller \  
--region ap-south-1
```

Delete K8s Resources

```
kubect1 delete -f ~/Complain_System/k8s/
```

Destroy Infrastructure

```
cd ~/Complain_System/terraform/municipal  
terraform destroy --auto-approve
```


DOCUMENT 2

How to Run the Project on EKS

Architecture Overview

Component	Purpose	AWS Service
Flask App (Gunicorn)	Web application server	EKS Pod
Database	Users, complaints, staff	RDS PostgreSQL
Image Storage	Complaint photos	Amazon S3
Notifications	Email on complaint submit	Amazon SNS
AI Model (.pkl)	Auto-assigns priority	Bundled in container
Image Registry	Stores Docker images	Amazon ECR
Load Balancer	Public entry point	AWS ALB (via ingress)

Prerequisites — Collect Before Starting

Item	Where to Get It
AWS Account with admin access	AWS Console
S3 Bucket ARN	Create in AWS Console → S3
SNS Topic ARN	Create in AWS Console → SNS
IAM Admin Role ARN	AWS Console → IAM → Roles
SSH Public Key	Run: <code>cat ~/.ssh/id_rsa.pub</code> on your machine
AWS Access Key (local only)	IAM → Users → Security credentials

Phase 1 — AWS Pre-Setup

Before running Terraform, create the AWS resources the application depends on.

- Create S3 bucket for complaint images: `aws s3 mb s3://YOUR_BUCKET --region ap-south-1`
- Create SNS topic: `aws sns create-topic --name demo --region ap-south-1` (save the ARN)
- Subscribe your email to the SNS topic via AWS Console → SNS → Topics → demo → Create subscription

- Create ECR repository: `aws ecr create-repository --repository-name complain-system --region ap-south-1`

Phase 2 — Deploy Infrastructure with Terraform

Clone the repo and edit only the terraform.tfvars file.

Clone and Configure

- Clone: `git clone https://github.com/mantu0tech/Complain_System.git`
- Navigate: `cd Complain_System/terraform/municipal`
- Edit terraform.tfvars with your values (see table below)

Variable	What to Change	Example
ssh_public_key	Paste output of: <code>cat ~/.ssh/id_rsa.pub</code>	"ssh-rsa AAAA..."
admin_role_arns.admin	Your IAM admin role ARN	"arn:aws:iam::051826716700:role/adminroleforproject"
db_password	Strong password (never commit)	"StrongPass123!"
instance_types	Use t3.medium for dev (cheaper)	["t3.medium"]

Run Terraform

1. `terraform init`
2. `terraform plan` (review what will be created)
3. `terraform apply --auto-approve` (takes ~15 minutes)
4. `terraform output` (save all values shown)

Key outputs to save: `jump_server_ip`, `eks_cluster_name`, `rds_endpoint`, `ssh_command`, `kubeconfig_command`

Phase 3 — Jump Server Setup

5. SSH in: `ssh -i ~/.ssh/municipal-key.pem ubuntu@YOUR_JUMP_SERVER_IP`
6. Install AWS CLI: `sudo apt install awscli -y`
7. Clone repo: `git clone https://github.com/mantu0tech/Complain_System.git && cd Complain_System`
8. Run install script: `bash install.sh` (installs Docker, kubectl, eksctl, Helm)

9. Attach IAM role via AWS Console: EC2 → Instances → municipal-jumpserver → Actions → Security → Modify IAM Role → admin_role
10. Configure kubectl: `aws eks update-kubeconfig --name municipal-dev --region ap-south-1`
11. Verify: `kubectl get nodes` (both nodes show Ready)

Phase 4 — Build and Push Docker Image

12. Create .env file on jump server: `nano .env` (use template from Document 1 Section 9)
13. Switch branch: `git checkout devops`
14. Login to ECR: `aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin YOUR_ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com`
15. Build: `docker build -t complain-system .`
16. Tag: `docker tag complain-system:latest YOUR_ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com/complain-system:latest`
17. Push: `docker push YOUR_ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com/complain-system:latest`
18. Verify: check AWS Console → ECR → complain-system → Images for latest tag

Phase 5 — Deploy to Kubernetes

19. `cd ~/Complain_System/k8s`
20. `kubectl apply -f namespace.yml`
21. `kubectl apply -f secret.yml`
22. `kubectl apply -f deployment.yml`
23. `kubectl apply -f service.yml`
24. `kubectl apply -f ingress.yml`
25. Verify: `kubectl get all -n municipal` (both pods show Running 1/1)

Phase 6 — AWS Load Balancer Controller

The ingress ADDRESS is empty until this controller is installed. It provisions the ALB and gives you a public URL.

26. `helm repo add eks https://aws.github.io/eks-charts` && `helm repo update`
27. Get VPC ID: `aws eks describe-cluster --name municipal-dev --region ap-south-1 --query cluster.resourcesVpcConfig.vpcId --output text`
28. Download IAM policy: `curl -o iam_policy.json https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/iam_policy.json`

29. Create IAM policy: `aws iam create-policy --policy-name AWSLoadBalancerIAMPolicy --policy-document file:///iam_policy.json`
30. Associate OIDC: `eksctl utils associate-iam-oidc-provider --cluster municipal-dev --region ap-south-1 --approve`
31. Create IAM service account (10-15 min): `eksctl create iamserviceaccount ...` (see Document 1 Section 7)
32. Install via Helm with your VPC ID (see Document 1 Section 7)
33. Restart: `kubectl rollout restart deployment aws-load-balancer-controller -n kube-system`
34. Get app URL: `kubectl get ingress -n municipal` (ADDRESS column = your public URL)

Phase 7 — Jenkins CI/CD

Every git push to master will automatically build, push to ECR, and deploy to EKS.

Install Jenkins

Follow Section 8 of Document 1. Access Jenkins at `http://YOUR_JUMP_SERVER_IP:8080`

Plugins to Install

Plugin	Purpose
Git	Checkout from GitHub
Pipeline	Declarative Jenkinsfile support
Docker Pipeline	docker build and docker push in pipeline
AWS Steps	AWS CLI integration
GitHub Integration	Receives webhooks from GitHub
Stage View	Visual pipeline view (optional)

Pipeline Job Configuration

- New Item → name: municipal-complaint → Pipeline → OK
- General: check GitHub project, enter repo URL
- Build Triggers: check GitHub hook trigger for GITScm polling
- Pipeline: Definition = Pipeline script from SCM, SCM = Git, Branch = */devops, Script Path = Jenkinsfile

GitHub Webhook

- Go to GitHub repo → Settings → Webhooks → Add webhook
- Payload URL: `http://YOUR_JUMP_SERVER_IP:8080/github-webhook/`

- Content type: application/json, Events: Just push events, Active: checked

DOCUMENT 3

Troubleshooting Guide

Quick Error Reference

Error	Cause	Fix
dial tcp 10.0.x.x:443: i/o timeout	EKS SG blocks port 443	Add HTTPS inbound from jump server SG to EKS cluster SG
ImagePullBackOff	ECR image not found or wrong URI	Check image URI in deployment.yml matches your ECR repo
CrashLoopBackOff	App crash — usually DB connection	kubectl logs POD_NAME -n municipal
no basic credentials	ECR token expired (12hr)	Re-run aws ecr get-login-password login command
EntityAlreadyExists	IAM policy already created	Skip creation, get ARN: aws iam list-policies
Tables created []	DB connected but no tables	Run db.create_all() via kubectl exec into pod
docker: permission denied	Jenkins not in docker group	sudo usermod -aG docker jenkins && restart Jenkins
ALB ADDRESS empty	LB controller not working	kubectl logs -n kube-system deployment/aws-load-balancer-controller
terraform force-unlock	State locked from crashed run	terraform force-unlock LOCK_ID
Pod Pending — no resources	Nodes too small/full	Increase node size in terraform.tfvars

Detailed Troubleshooting by Phase

AWS CLI — Unable to locate credentials

Error: Unable to locate credentials

Fix: Run aws configure and enter your Access Key ID, Secret Access Key, Region (ap-south-1), Output format (json).

S3 — BucketAlreadyExists

Error: An error occurred (BucketAlreadyExists)

Fix: Bucket names are globally unique. Add a random suffix to your bucket name, e.g. your-s3-bucket-complain-2024.

S3 — InvalidBucketName

Error: The specified bucket is not valid

Fix: Bucket names must be all lowercase, no underscores. Use hyphens only:
your-s3-bucket-complain

Terraform — Error acquiring the state lock

Error: Error locking state: Error acquiring the state lock

Fix: A previous run crashed and left the lock. Run:

```
terraform force-unlock LOCK_ID
```

Terraform — Apply takes over 30 minutes

EKS creation normally takes 12-15 minutes. If over 30 minutes:

- Check AWS Console → CloudFormation for stacks in ROLLBACK status
- Check AWS Console → EKS → Clusters for error messages
- Cancel and re-run: terraform destroy --auto-approve then terraform apply --auto-approve

Terraform — Unsupported instance type

Error: InvalidParameterException: Unsupported instance type m7i-flex.large

Fix: Change instance type in terraform.tfvars:

```
instance_types = ["t3.medium"]
```

SSH — Permission denied (publickey)

Error: Permission denied (publickey)

Fix: Check key file permissions and use the correct key:

```
chmod 400 ~/.ssh/municipal-key.pem  
ssh -i ~/.ssh/municipal-key.pem ubuntu@YOUR_JUMP_SERVER_IP
```

SSH — Connection timed out

Error: ssh: connect to host xxx.xxx.xxx.xxx port 22: Connection timed out

Fix: The jump server security group is blocking port 22. Go to AWS Console → EC2 → Security Groups → municipal-jumpserver-sg → Inbound Rules → Add SSH from your IP.

kubect! — dial tcp 10.0.x.x:443: i/o timeout

Error: couldn't get current server API group list: dial tcp 10.0.x.x:443: i/o timeout

This is the most common error. The jump server security group cannot reach EKS on port 443.
Fix:

35. AWS Console → EKS → Clusters → municipal-dev → Networking
36. Click the Cluster security group link (sg-xxxxxxxxxx)
37. Click Edit inbound rules
38. Add rule: Type = HTTPS, Protocol = TCP, Port = 443, Source = your jump server security group (municipal-jumpserver-sg)
39. Click Save rules
40. Run `kubectl get nodes` again

Alternative fix via CLI:

```
# Get jump server SG ID
aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=municipal-jumpserver" \
  --query "Reservations[*].Instances[*].SecurityGroups[*].GroupId" \
  --output text

# Get EKS cluster SG ID
aws eks describe-cluster \
  --name municipal-dev --region ap-south-1 \
  --query cluster.resourcesVpcConfig.clusterSecurityGroupId \
  --output text

# Add the rule
aws ec2 authorize-security-group-ingress \
  --group-id EKS_CLUSTER_SG_ID \
  --protocol tcp --port 443 \
  --source-group JUMPSEVER_SG_ID \
  --region ap-south-1
```

Pod — ImagePullBackOff or ErrImagePull

Error: Failed to pull image: not found or access denied

Diagnosis:

```
kubectl describe pod POD_NAME -n municipal # look at Events section
```

Fixes:

- Check image URI in deployment.yml exactly matches your ECR repository URI including account ID
- Ensure EKS node IAM role has AmazonEC2ContainerRegistryReadOnly policy attached
- Confirm the image was pushed: `aws ecr describe-images --repository-name complain-system --region ap-south-1`

Pod — CrashLoopBackOff

Error: Back-off restarting failed container

Diagnosis:


```
kubectll logs POD_NAME -n municipal
```

Common causes and fixes:

- DATABASE_URL wrong — check secret.yml has correct RDS endpoint (not db:5432)
- RDS not reachable — check RDS security group allows port 5432 from EKS node security group
- Missing .pkl model file — ensure priority_model.pkl and vectorizer.pkl are in the Docker image
- SECRET_KEY not set — check secret.yml has all required environment variables

Pod — Pending (no resources)

Error: 0/2 nodes are available: Insufficient cpu/memory

Fix: Increase node size or count in terraform.tfvars:

```
eks_node_groups = {  
  default = {  
    desired_size    = 3  
    instance_types = ["t3.large"]  
  }  
}
```

Then: terraform apply --auto-approve

DB Tables Not Created

Tables created [] (empty list — DB connected but no tables exist)

Fix: Exec into a running pod and create tables manually:

```
kubectll exec -it -n municipal POD_NAME -- python -c \  
"from app import app, db; app.app_context().push(); db.create_all();  
print('Tables created')"
```

ECR — no basic credentials

Error: no basic credentials

ECR login tokens expire after 12 hours. Re-authenticate:

```
aws ecr get-login-password --region ap-south-1 | \  
docker login --username AWS --password-stdin \  
YOUR_ACCOUNT_ID.dkr.ecr.ap-south-1.amazonaws.com
```

Then retry the docker push command.

ECR — access denied / not authorized

Error: User is not authorized to perform: ecr:GetAuthorizationToken

Fix: The IAM role is missing ECR permissions. Go to IAM → Roles → your jump server role → Attach policies → Add AmazonEC2ContainerRegistryFullAccess.

ALB — ingress ADDRESS stays empty

kubect! get ingress -n municipal shows ADDRESS column blank after 5+ minutes

Check controller logs:

```
kubect! logs -n kube-system deployment/aws-load-balancer-controller
```

Common causes:

- VPC ID wrong in helm install command — re-check aws eks describe-cluster output
- Subnets missing kubernetes.io/role/elb=1 tag — check VPC module in Terraform
- IAM service account not created correctly — re-run eksctl create iamserviceaccount step
- Policy ARN wrong — verify account ID in the policy ARN used in eksctl command

Jenkins — docker: Got permission denied

Error: Got permission denied while trying to connect to the Docker daemon socket

```
sudo usermod -aG docker jenkins  
sudo systemctl restart jenkins
```

Jenkins — kubect!: no config / no such file

Error: error: no configuration has been provided

```
sudo cp ~/.kube/config /var/lib/jenkins/.kube/config  
sudo chown -R jenkins:jenkins /var/lib/jenkins/.kube  
sudo systemctl restart jenkins
```

Jenkins — Webhook Not Triggering

Pushing to GitHub does not start a Jenkins build

Check these in order:

- Jump server SG allows port 8080 inbound from 0.0.0.0/0
- Webhook URL in GitHub ends with /github-webhook/ (trailing slash required)
- GitHub Integration plugin is installed in Jenkins
- Check delivery status: GitHub repo → Settings → Webhooks → click webhook → Recent Deliveries (green tick = working)

Jenkins — Build Stuck

Jenkins build shows running but no output for many minutes

Add timeout to Jenkinsfile (already included in the dynamic Jenkinsfile from this guide):

```
options {  
    timeout(time: 30, unit: 'MINUTES')  
}
```

eksctl — EntityAlreadyExists on IAM policy

Error: EntityAlreadyExists: A policy called AWSLoadBalancerIAMPolicy already exists

The policy was created in a previous attempt. Skip the create-policy step and get the existing ARN:

```
aws iam list-policies \
  --query "Policies[?PolicyName=='AWSLoadBalancerIAMPolicy'].Arn" \
  --output text
```

eksctl — AlreadyExistsException on CloudFormation

Error: AlreadyExistsException: Stack already exists

A previous service account creation partially succeeded. Delete the stack and retry:

```
aws cloudformation delete-stack \
  --stack-name
eksctl-municipal-dev-addon-iamserviceaccount-kube-system-aws-load-balancer-controller
```

Wait 2 minutes, then re-run the eksctl create iamserviceaccount command.

IMPORTANT: The AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY visible in the PDF screenshots are from a demo environment and have been rotated. Never commit credentials to git or include them in Docker images.

End of Document